



Better reasoning about software engineering activities

Tim Menzies¹
University of British Columbia
tim@menzies.com

James D. Kiper²
Miami University
kiperdj@muohio.edu

ABSTRACT¹

Incomplete information may prevent software analysts from making categorical decisions about software. This problem is particularly acute when discussing subjective early life cycle issues in data-starved domains that lack an experience repository. This paper discusses JANE- a language for better reasoning about subjective information in data starved domains. JANE uses *stochastic search* to generate logs of behavior from models of subjective knowledge and *inductive learning* to summarize those logs. JANE can be used for (i) cost optimization, (ii) the semi-automatically analysis of a very large number of choices; and (iii) handling uncertainty about the extent to which sub-goals influence each other. In experiments with logs generated from an encoding of the SEI CMM level-2, JANE could find a small number of *control actions* that could better achieve top-level goals of some software development task.

LENGTH: 5900 words

1. Introduction

Why is data mining not used more in software engineering (SE)? Recent reviews of the use of data mining in SE (e.g. [15, 16]) report that data mining in SE is a mature technique based on widely-available tools using well understood algorithms (e.g. neural nets or decision tree learners or Bayes nets [3, 8, 11, 24]). Also, impressive results have been reported in certain domains (e.g. cost-estimation [3] or prediction of faulty modules [28]). Further, data mining tools that can handle very large data sets now come bundled and integrated with standard commercial packages such as Microsoft's SQL[®] Server 2000TM [26]. Lastly, clear and simple methodological guidelines for data mining in SE have existed for nearly a decade [22]. Nevertheless, the total number of reported applications (as seen in [15, 16]) is not large.

One explanation for the lack of reported applications is the poor state-of-the-art in SE data collection. Data miners

need data and, often, there has not been much data available. For example, the majority of software development organizations do not conduct systematic data collection. As evidence for this, consider the Software Engineering Institute's capability maturity model (CMM [20]) which categorizes software organizations into one of five levels based on the maturity of their software development process. Below CMM level 4, there is no systematic data collection. Below CMM level 3, there is not even a written definition of the software process. Many organizations exist below CMM level 3² hence reliable data on SE projects is scarce, or hard to interpret.

Another explanation is that standard data mining answers the wrong question. A standard output from a data miner is a *classifier* which converts some inputs into one of a small number of distinct classes (e.g. *lowFaultModule*, *highFaultModule*). Such a result may not be of interest to a software manager. As one manager told us:

Don't tell me I am heading for a cliff- rather, tell me how to avoid this cliff NOW and all cliffs like that in the future.

That is, software managers are less interested in *assessment models* which report the status of a current situation than in *controller models* for changing the current and future situation.

When we protested to our software manager that we don't have the data required to answer her question, she posed an alternate goal for data mining which we call the *minimal management question*:

The minimal management question: *If you can't tell me what to do, can you at least tell me what is not worth doing?*

That is, given the huge range of tools and methodologies available in the commercial world, our manager wants to avoid wasting time on techniques that are not useful.

For example, Figure 1 shows the impacts of three different sets of management decisions on a software project. The

¹Note to reviewers: the abstract originally submitted to ASE2001 stressed the comparative merits of this approach to the softgoals of Chung et.al. [4] However, as the paper matured, other aspects became more important and softgoals are now only mentioned in the Related Work section.

¹Corresponding author. Department of Electrical & Computer Engineering; 2356 Main Mall; Vancouver, B.C. Canada V6T1Z4. Phone: (604) 822-3381 Web: <http://tim.menzies.com>.

²Dept. Computer Science & Systems Analysis, Miami University, Oxford, Ohio, USA.

²Personal communication with SEI researchers.

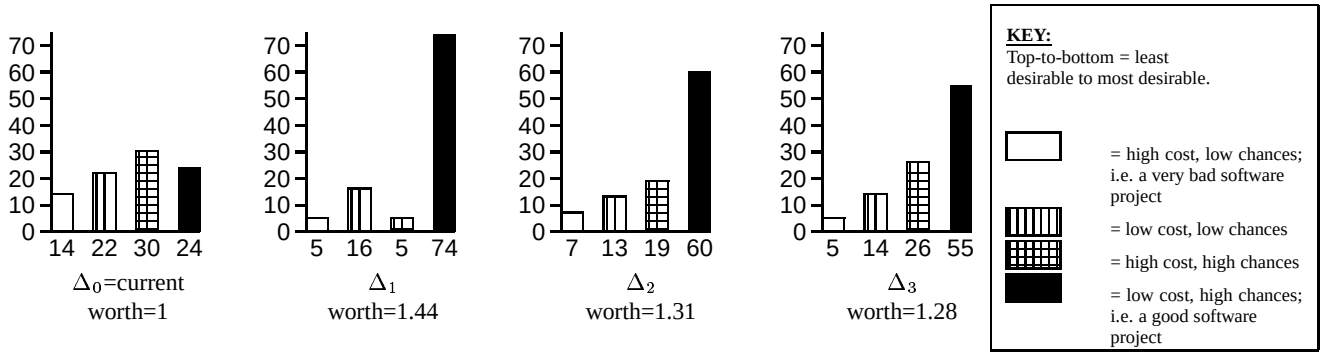


Figure 1. Ratios of different software project types seen in four situations.

left-hand-side histogram marked Δ_0 shows the ratio of different project types predicted by a software management oracle (described below). The other histograms ($\Delta_{i>0}$) show how those ratios change after applying *treatments*; i.e. managers taking certain actions to change their current situation. The *worth* of each option is a reflection of the proportion of good and bad projects, compared to Δ_0 ; i.e. ($worth(\Delta_0) = 1$). Note that as *worth* increases, the proportion of preferred projects also increases.

Figure 2 shows some of the potential treatments known to the software management oracle. The underlined treatments in that figure show the treatments used in Figure 1. Note that most of the treatments are not underlined; i.e. many treatments can be proposed in this domain, but only a very small subset appear in the best treatments shown in Figure 1. Figure 2 is hence an answer to the minimal management question since it identifies ignorable management actions.

The rest of this paper describes how Figure 1 was generated using *stochastic simulation* followed by *summarization*. The software management oracle used in the this study contains numerous subjective features. At each subjective point, a range of behaviors is possible. Each stochastic simulation samples one possible behavior. After many such stochastic simulations, it was possible to find treatments that had (usually) the same impact despite the variability in the oracle. These treatments were found by summarizing a log of the simulations with the TAR2 *treatment learner*. This paper presents the first use of TAR2 on a real-world model (previous reports were focused on toy examples [17]).

2. Building a Software Management Oracle

This section describes the software management oracle used in the study. Three components are required for such an oracle. Firstly, a knowledge source is required. This study used a detailed description of CMM2 [21, p125-191]. We used CMM2 since, in our experience, many organizations can achieve at least this level. CMM2 is less con-

cerned with issues of (e.g.) which design pattern to apply, than with what overall project structure should be implemented. Improving CMM2-style decisions is important since in early software lifecycle, many CMM2-style decisions effect the resource allocation for the rest of the project.

Secondly, a syntax is required to encode the knowledge source. This study uses the syntax of the JANE language [17] which is an extension to the ARRT system of Feather et.al. [7].

Thirdly, an interpreter is required to execute the syntax. In the case of subjective software engineering knowledge, this interpreter must be able to handle *degrees of belief*. Hence, the system described below assigns a Chances weight to all of its propositions.

The core data structure in ARRT is a network of *faults* and risk mitigation *actions* that effect a tree of *requirements* written by the stakeholders. Potential faults within a project

<u>baselineAudits,</u>	<u>planRevised,</u>
<u>baselineChangesControlled,</u>	<u>requirementsReview,</u>
<u>changeRequestsHandled,</u>	<u>requirementsUsed,</u>
<u>changesCommunicated,</u>	<u>reviewRequirementChanges,</u>
<u>configurationItemStatus-</u>	<u>risksTracked, SCMplan,</u>
<u>Recorded,</u>	<u>SCMplanUsed,</u>
<u>deviationsDocumented, docu-</u>	<u>SElifeCycleDefined, SEteam-</u>
<u>mentedDevelopmentPlan,</u>	<u>ParticipatesInPlanning,</u>
<u>documentedProjectPlan,</u>	<u>SEteamParticipatesOnPro-</u>
<u>earlyPlanning,</u>	<u>posal, SQAauditsProducts,</u>
<u>formalReviewsAtMilestones,</u>	<u>SQAplan, SQAplanUsed,</u>
<u>goodUnitTesting,</u>	<u>SQAreviewActivities,</u>
<u>identifiedWorkProducts,</u>	<u>workProductsIdentified</u>
<u>periodicSoftwareReviews</u>	

Figure 2. Management actions in the CMM2 model. SQA= software quality assurance and SCM= software configuration management)

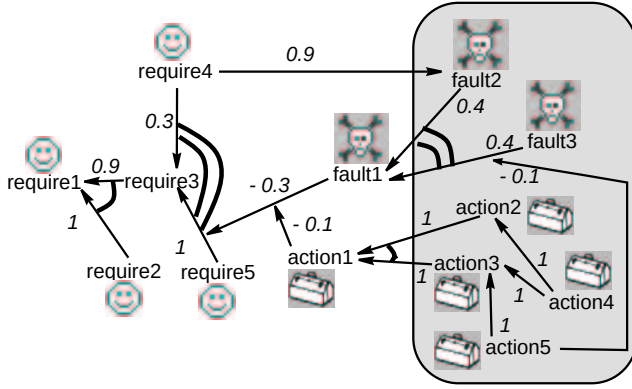


Figure 3. Left: an An ARRT/JANE-style [7] software management oracle. Right: explanation of symbols.

are modelled as influences on the edges between requirements. Potential fixes are modelled as influences on the edges between faults and requirements edges.

In ARRT, (i) a requirement is reachable if the disjunction of any of its children are reachable; (ii) faults and actions are flat facts. Users of ARRT sometimes find these limitations to restrictive. For example:

- Hardware engineers often use fault trees.
- Software process engineers often describe processes with some sequence to them; e.g. CMM-4 can't be implemented till after CMM-3.

It would be useful if ARRT could be integrated with these data structures. To support this, the JANE language allows faults, actions, and requirements to exist in arbitrarily complex networks of conjunctions and disjunctions. Figure 3 shows what can be expressed in ARRT and JANE: the section marked with an oval is not expressible in ARRT and contains dependencies between actions, between faults, between actions and faults, and between requirements and faults.

ARRT-style RE outputs a minimum set of “risk-mitigations”, a.k.a. treatments, which address the potential faults. Specifically, the goal of a treatment is to *maximize* our coverage of the requirements while *maximizing* the ways the actions reduce the impact of the faults and *minimizing* the costs of the actions. Optimizing on all these criteria is complicated by the interactions inside the model. For example, in Figure 3, *fault2* and *require4* are interconnected: if we cover *require4* then that makes *fault2* more likely which, in turn, makes *fault1* more likely which reduces the contribution of *require5* to *require3*. Optimization within JANE is performed by summarizing multiple *random searches* over (e.g.) Figure 3 to find the key values that most effect the system. This summarization process is

defined later, after a tour of JANE.

2.1. JANE

A JANE programmer enters in models in a propositional rule-based language of the form: conclusion if precondition.

Internally, these rules are converted into a directed graph with the following BNF³:

```
Graph ::= Goal (Vertex)* (Edge)*
Goal  ::= Item
Vertex ::= Mix | Item
Item   ::= Type Variable Value Label
Type   ::= action | fault | requirement
Mix    ::= CombineRules Order
```

Note that each Graph has a special Goal Item. Conceptually, JANE is a backward chainer that performs a recursive descent from the Goal.

In JANE, every vertex is either:

- An Item where some Value is assigned to some Variable.
- Or a Mix vertex where influences are combined.

Before we can explain Mixing, we must explain what is mixed: i.e. Costs and Chances.

2.2. Cost and Chances

Each Vertex and Edge in JANE is augmented with a Cost and a Chances weight:

```
Edge  ::= Vertex Label Vertex
Label ::= Cost Chances
Cost  ::= null | 0.00 .. infinity
Chances ::= null | 0.00 .. 1.00
```

JANE's Chances define the extent to which a belief in one vertex can propagate to another. Costs let an analyst model

³In this article's BNF notation, $W ::= X Y | Z$ denotes that the structure W contains either the structures X and Y or the structure Z . $(X)^*$ denotes zero or more repeats of X . $(X)^+$ denotes one or more repeats of X . $[X]$ denotes that X is optional. Terminals start with lower case, or are quoted

the common situation where some of the **Cost** of some procedure is amortized by reusing its results many times. Hence, the *first* time we use an **action** we incur that **Cost** but afterwards, that **action** is free of charge.

2.3. Combine Rules

The **Cost** and **Chances** of **Edges** are provided by the JANE programmer. The **Cost** and **Chances** of **Vertices** are either provided by the JANE programmer or computed at runtime via a traversal of the edges. In JANE, this computation is defined by the **CombineRules** at the **Mix** vertices. For each child of a **Mix** node, a recursive descent is executed and the returned **Cost** and **Chances** values are combined according to the **CombineRules**:

```
CombineRules ::= CostCombine
              ChancesCombine
CostCombine  ::= first(cost) | sum(cost)
ChancesCombine ::= first(chances) |
                  | sum(chances) negate
                  | product(chances)
```

Negate is used for negation. For example, when searching *X if not A*, the **Chances** of *X* are $1 - \text{Chances}(A)$.

Product(chances) is used for *conjunctions*. For example, when searching *X if A and B and C*, the **Chances** of *X* is the product of the chances of *A, B, C*.

First(X) is used for *simple disjunctive evidence*. For example, when testing *X if A or B or C*, then the **Cost** and **Chances** of *X* is taken from the first member of *A, B, C* that is satisfied.

Sum(X) is used for *summing disjunctive evidence*. For example, JANE supports a special operator called **ors** that is used for implementing disjunctive summing (JANE also supports several other novel operators, described in the next section). **Ors** is like **or** except that when testing (e.g.) *X if A ors B ors C*, all members of *A, B, C* will be tested. If at least one succeeds, the **Cost** and **Chances** of *X* is summed from the satisfied members of *A, B, C*.

Summing evidence is useful when several weak supports for a goal can sum to a strong belief in that goal. JANE supports both **or** and **ors** since summing is valid in some cases and invalid in others. Independent indicators such as *moving* and *talking* could be summed to form a stronger belief such as *alive*. Summing dependent indicators such as *hasBloodPressure* and *heartBeating* implies *alive* is deprecated.

Sum(X) and **Product(X)** can be combined to implementing mitigation effects. For example, recall from Figure 3 that *action5* disables most of the contribution of *fault3* onto *fault1*. In the JANE syntax, this can be coded as *fault1 if*

```
    fault2 @ 0.4
    ors (fault3 @ 0.4
        and not action5 @ 0.9)
```

The **ors** operator uses **Sum(chances)**; and uses **Product(chances)**; and not uses **negate**. Hence, if the **Chances** of *fault2, fault3, action5* are all unity, then:

- If *action5* is false, the **Chances** of *fault1* is $0.4 + (0.4 * (1 - 0)) = 0.8$.
- If *action5* is true, the **Chances** of *fault1* is $0.4 + (0.4 * (1 - 0.9)) = 0.44 \ll 0.8$.

That is, using *action5* significantly reduces the **Chances** of *fault1*.

2.4. JANE and Random Search

JANE supports two mechanisms for random search. Firstly, when defining **Costs** and **Chances**, the programmer can supply a *range* and a *skew*. For example: *goodUnitTesting* and *cost = 1 to +5*

defines the *cost* of *goodUnitTesting* as being somewhere in the range 1 to 5, with the mean skewed slightly towards 5 (denoted by the “+”). During a simulation, the first time this cost is accessed, it is assigned randomly according to the range and skew. The assignment is cached so that all subsequent accesses use the same randomly generated value. After each simulation, the cache is cleared. After thousands of simulations, JANE can sample the “what-if” behavior resulting from different assignments within the range.

Secondly, JANE can randomized where it searches using the **Order** of the **Mix** operators:

```
Order      ::= Random | Left2Right
Left2Right ::= and | or | ors | any | not
Random     ::= rand | ror | rors | rany
```

In the case where the child-order represents some sequence that must be persevered (e.g. ordering software processes) JANE programmers can use **Left2Right** operators. However, repeatedly searching a graph in the same order may miss certain features. For example, in Figure 3, either “*action4*” or “*action4 and action5*” could be used to achieve *action1*. Assuming all actions cost the same, then ignoring *action5* and merely performing *action4* would be cheaper method of achieving *action1*. However, this would imply that *action5* would no longer be disabling most of *fault3*.

In order to not miss such important interactions, JANE supports several random search operators: **rand**, **ror**, **rors**, and **rany**.

- For $op \in \{rand, ror\}$, JANE tries to prove all of *X op Y op Z*, but does so in some randomly selected order.
- **Rany** is best understood by comparison with **ror**. The expression *X ror Y ror Z* is exited when any one of *X, Y, Z* is satisfied. In contrast, JANE tries to prove one or more of *X rany Y rany Z*, and does so in some randomly selected order. **Rany** is useful when searching for subsets that contribute to some conclusion. For

example, the following JANE rule offers several essential features of *stableRequirements* plus several optional factors relating to monitoring change in evolving projects. The essential features are *rand*-ed together while the optional factors are *rany*-ed together.

```

stableRequirements if effectiveReviews @ 0.3
  rand requirementsUsed @ 0.3
  rand sEteamParticipatesInPlanning @ 0.3
  rand documentedRequirements @ 0.3
  rand sQAactivities @ 0.3
  rand (reviewRequirementChanges @ 0.3
    rany softwareConfigurationManagement@0.3
    rany baselineChangesControlled @ 0.3
    rany workProductsIdentified @ 0.3
    rany softwareTracking @ 0.3).

```

- **Rors** is useful for specifying the high-level goals of the system. While *rany* will search *some subset* of its parameters, *rors* will search *all* its parameters. For example, in this less-than-perfect world, it is unlikely we can be *rich* and *happy* and *healthy*, but that should not stop us from trying; i.e.
 goal if rich rors happy rors healthy

The operators *rand*, *ror*, *rors* and *rany* have the same satisficing criteria as *and*, *or*, *ors* and *any* respectively, but the latter search left-to-right while the former search in a randomly selected order. In terms of the **CombineRules** described above, *And* and *rand* are *conjunctive* operators; *Or* and *ror* are *simple disjunctive* operators; and the rest are *summing disjunctive* operators.

2.5. The CMM2 Model

CMM2 written in JANE has 55 **Items** with **Value** $\in \{t, f\}$. Of the 55 **Items**, 27 were identified as management actions that could be changed by managers (see Figure 2). The top-level **Goal** was *goodProject*.

Within the model, **Chances** values were added to 79 edges of the 150 edges in the model. While each of these values are based on expert judgement, their precise value is subjective. Hence, each such **Chances** value X was altered to be a range
 chances = $0.7 * X$ to $1.3 * X$

so the simulator could experiment with values nearby the stated **Chances** value.

The full model is available via email from the authors.

3. Learning From JANE

One execution of JANE can be summarized by the **Cost** and **Chances** of the top-level goal *goodProject*, the 79 values assigned to the subjective edges, and the values assigned to the 27 manager's actions of Figure 2. Random walking over JANE models can hence generate an overwhelming amount of data (79+27+2=108 data points per simulation, times the number of simulations).

Summarization of JANE output is performed by the TAR2 *treatment learner* [17]. A treatment learner outputs a set of possible *treatments* (Δ) using the process described below. Each treatment is assessed by the *change* it makes to the distribution of classes. A *good* treatment increases the frequency of the *better classes* (where “better” is defined below).

Treatment learners input a set of classified examples (*Eg*). Each example is a conjunction of attribute ranges, plus one classification c . This classification comes from a pre-defined set of classes C ; i.e. $c \in C$. Each classification c is associated with a numeric *score* i.e. some classifications score better than others.

For the CMM model, after 2000 random walks, a wide range of generated **Costs** and **Changes** in *good-Project* were observed. These ranges were sub-divided into high/low bands of roughly the same size. Combining high/low **Cost/Chances** yields four classes. These classes were scored as follows:

Score=0: *High cost, low chance*

Score=1: *Low cost, low chance*

Score=2: *High cost, high chance*

Score=4: *Low cost, high chance*.

That is, our preferred projects are cheap and highly likely while expensive, low odds projects are to be avoided.

A treatment learner searches for *candidate* attribute ranges; i.e. ranges that are more common in the best classification than in the not-so-best classification. In the CMM domain, such a candidate is an attribute range that would tend to drive the system into *low cost, high chance projects*.

A heuristic ranking δ is given to each candidate reflecting (i) how much more common is the candidate in the best class than in some non-best-class c ; and (ii) how much better is the best class than class c :

$$\delta = (score(best) - class(c)) * \left(\frac{|Eg \cap A = R \cap class = best|}{|Eg \cap A = R \cap class = c|} \right)$$

where $A = R$ denotes an attribute A with range R and $|Eg \cap X|$ denotes the size of the subset of examples that contain X .

Figure 4 shows the frequency counts of the δ s seen in 2000 runs of the CMM-2 model. Candidates with a large δ score are attribute ranges that are far more frequent in the best class (*high chances, low cost projects*) than in other classes. Looking at the right-hand-side of Figure 4, we see that there exist a small number of candidates with outstandingly large δ s. These are the attribute ranges that occur far more frequent in the best class (*high chances, low cost projects*) than in other classes. The *treatments* Δ are all subsets of candidates with a ranking higher than some user-specified δ threshold value δ_T . Looking at Figure 4, at $\delta_T = 8$, there $10+4+2=16$ candidates.

In reality, it is hard to change many aspects of a project and the wise software engineer seeks the smallest number of

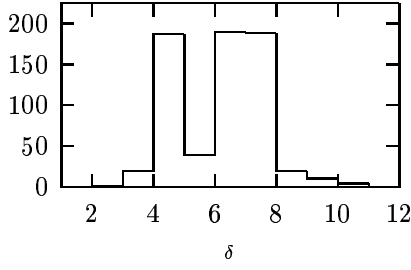


Figure 4. Frequency counts of δ s seen in the CMM-2 model.

changes with the greatest impact. Assuming we seek three changes, there are 560 possible treatments of size 3 in out 16 candidates. The *best* treatment is the one that offers the biggest improvement to the class distributions seen in the untreated data. To find the best treatment, we compute following ratio for all 560 treatments:

$$worth(\Delta_i) = \frac{\sum_{c \in C} |Eg \cap \Delta_i \cap class = c| * score(c)}{baseline\ worth} \quad (1)$$

where *baseline worth* is the *worth* seen in the untreated example set:

$$baseline\ worth = \sum_{c \in C} |Eg \cap class = c| * score(c)$$

Figure 5 shows the three best treatments ($\Delta_1, \Delta_2, \Delta_3$) found using this technique (and Figure 1 compared the effects of these treatments to the untreated examples). Note that the values of each attribute are reported using the tags *no*, *lower*, *middle*, or *upper*. In treatment learning, continuous attribute ranges are divided into N-descrete bands based on percentile positions. For N=3, we can name the bands *lower*, *middle*, *upper* for the lower, middle, and upper 33% percentile bands.

In Figure 5, Δ_2 and Δ_3 are advising to lower the cost of: **Using requirements:** This could be accomplished by (e.g.) by sharing them around the development team in some searchable hypertext format

Performing formal reviews at milestones: This could be accomplished by (e.g.) using ultra-lightweight formal methods such as proposed by Leveson [14].

Performing good unit testing: This could be accomplished by (e.g.) hiring better test engineers.

The value *no* refers to missing values; a report of $X=no$ is a recommendation *not* to use X as part of a treatment. Such a negative recommendation is seen in Δ_1 which is advising against *periodicSoftwareReviews* (plus lowering the costs of

Δ_1 : *requirementsUsed.Cost=lower* and *not periodicSoftwareReviews=no* and *formalReviewsAtMilestones.Cost=lower*

Δ_2 : *requirementsUsed.Cost=lower* and *goodUnitTesting.Cost=middle* and *formalReviewsAtMilestones.Cost=lower*

Δ_3 : *goodUnitTesting.Cost=lower* and *periodicSoftwareReviews.Cost=middle* and *formalReviewsAtMilestones.Cost=lower*

Figure 5. The three best treatments found in the CMM-2 model.

using requirements and formal reviews at milestones). Note that if *periodicSoftwareReviews* are conducted, Δ_3 is saying that there is no apparent need to reduce the cost of such reviews.

The minimal management question in the introduction can now be answered. The underlined terms in Figure 2 are the actions found in the best treatments. The un-underlined terms in Figure 2 are the factors that were not found to usually change the **Cost** and **Chances** of a *goodProject* and can be ignored by management.

4. External Validity

There are several threats to the external validity of the above conclusions. Firstly, while the knowledge source used in this study (CMM2) is widely accepted, it is hardly universally accepted. Proponents of some other knowledge source could reject the specifics of the above conclusions, but still use the general technique; i.e. they could encode their preferred knowledge source in JANE and repeat the above analysis.

Secondly, numerous subjective choices were made when encoding JANE. While these choices were made using our best expert judgement, it is possible that our version of CMM2 is somehow different to the intent of the originally CMM2. Fortunately, JANE gives us two facilities for recognizing and handling such anomalies:

- Figure 1 is a summary of the stable properties seen in 2000 runs of our encoding of CMM2. Our model of CMM2 could be faulted if that summary does not match expert intuitions in this domain. In fact, the JANE method was originally devised as a validation tool for subjective knowledge; i.e. to test subjective knowledge, sample and summarize it's behavior.
- If disputes arise between JANE programmers on the appropriate encoding, all the possible resolutions to the dispute could be added to a JANE program. If, after thousands of executions, these different resolutions have no net impact on the overall behavior of the system (as summarized by displays such as Figure 1), then we need not resolve those particular disputes.

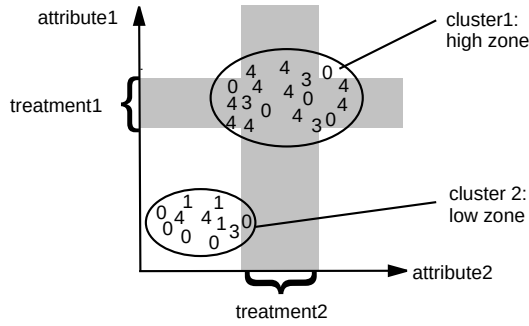


Figure 6. Treatments in attribute space.

Thirdly, even if results like Figure 1 are applicable to general software development, a particular project may have idiosyncracies that distance it from most other projects. To handle such idiosyncracies, JANE lets a programmer provide an initial set of fixed `Variable`, `Value` and `Label` bindings to a `Vertex`. Such initial bindings restrict the simulation to just executing scenarios relevant to a particular project and, hence, restrict the generated treatments to those relevant to a project.

Fourthly, the above conclusions were inferred from a random sampling of the possible behavior in the CMM-2 model. One issue with random sampling is *coverage*; i.e. the random search may not have found everything of interest in the model. To protect against this, it is good practice in treatment learning to double the size of the sample and check that the treatments learnt from a sample of size S still hold in a sample of size $2S$. At 4000 runs of the CMM-2 model, only the candidates underlined in Figure 2 were in the top 3 treatments.

5. Related Work

Bayesian reasoning has been used to sketch out subjective knowledge (e.g. our software management oracle), then assess and tune that knowledge based on available data. Success with this method includes the COCOMO-II effort estimation tool [3] and defect prediction modelling [8]. In domains that lack the data required for the tuning (e.g. our test domains), Bayesian reasoning is impractical.

Treatment learners solve a problem that is subtly different from the standard machine learning problem solved by (e.g.) C4.5 [23]. Standard learners assume that all classes are of equal value while treatment learners assume that some partial ordering exists between classes. Standard learners output some descriptor of the clusters in attribute space e.g. the circles on Figure 6. Treatment learners output a treatment which select for examples from higher scoring classes; e.g. the two treatments shown on the x-axis and

y-axis of Figure 6. Note that a prerequisite for treatment learning is that some attributes are *control variables*; i.e. they have some causal influence on the system. Candidate control variables for CMM2 were shown in Figure 2.

Our work could be characterized as lightweight requirements engineering for software process decisions. *Goal hierarchies* are a technique for heavyweight requirements engineering about software product decisions [29]. A goal hierarchy is a hierarchy of goals where every node contains assertions about products in the domain, expressed as fragments in the UML syntax. As system goals change, parts of this hierarchy are pruned and the implemented system is designed from the remaining UML fragments. Artifacts within the goal hierarchy are described in sufficient detail to permit a formal analysis of (e.g.) the invariants within a system. Goal hierarchies are inappropriate for subjective information such as our software management oracle since formal methods do not permit degrees of belief in some proposition. Also, the detail required for formal approaches may be unavailable, especially in the early parts of the software lifecycle.

When full formal methods is impractical, an alternative is to use formal tools on highly abstracted versions of the domain models. For example, Leveson [13] argues that the cost-effective safety testing of requirements can be accomplished via an analysis of a lightweight model such as fault trees [14] or state machines [10]. Both heavyweight and lightweight formal tools require a *systems description* and some *formal properties* to prove across those systems. In the software management literature we have seen, and in our own work, the only property sought has been coverage of some high-level goals. Such simple coverage properties may not require the intricacy of formal tools (applied either in a lightweight or a heavyweight mode).

Most formal methods focus on *product* features of software. An alternate approach are process programming languages such as Little-JIL [2, 30]. In Little-JIL, each software task can include pre-conditions, post-conditions, exception handlers, and sub-tasks. The core data structure of Little-JIL is hence a top-down decomposition tree showing sub-tasks inside tasks. An interpreter exists for Little-JIL models [2] but in the available reports⁴, there seems no emphasis of the processing of subjective knowledge.

The *softgoal framework* of Chung, Nixon, Yu, and Mylopoulos [4] (hereafter, *CNYM*) was especially designed for the processing of subjective knowledge. In CNYM, *softgoals* are distinguished from *goals* as follows. A *goal* has a well-defined non-optional feature of a system that *must* be available. A *softgoal* is goal that has no clear cut criteria for success. While goals can be conclusively demonstrated to be *satisfied* or *not satisfied*, softgoals can only be *satisfied* to some qualitative degree. Softgoal *label propagation rules* define how these qualitative values impact each other

⁴I.e. [2, 30] and the papers accessible from <http://laser.cs.umass.edu/tools/littlejil.html>

(JANE performs the same task using its `CombineRules`). In CNYM, when two qualitative influences conflict, CNYM requests a resolution for the user. When the softgoal models are small, when the stakeholders all agree, and the issues are clear-cut, then “ask the user” is a valid method of probing a space of subjective options. However, when models get large or more intricate, or when the stakeholders all argue about which models or goals are appropriate, then manually selected resolutions to individual conflicts become difficult. JANE enables the exploration of multiple sets of possible resolutions via numerous stochastic simulations.

Other research has explored simulation for making design decisions. Bricconi et.al. [9] built a simulator of a server on a network, then assessed different network designs based on their impact on the server. Menzies and Sinsel assessed software project risk by running randomly selected combinations of inputs through a software risk assessment model [19]. Josephson et.al. [12] executed all known options in a car design to find designs that were best for city driving conditions. Bratko et.al. [1] built qualitative models of electrical circuits and human cardiac function. Where uncertainty existed in the model, or in the propagation rules across the model, a Bratko-style system would backtrack over all possibilities.

Simulation is usually paired with some summarization technique. Our research was prompted by certain short-comings with the summarization techniques of others. Josephson et.al. used *dominance filtering* (a Pareto decision technique) to reduce their millions of designs down to a few thousand options. However, their techniques are silent on automatic methods for determining the difference between dominated and undominated designs. Bratko et.al. used standard machine learners to summarize their simulations. Menzies and Sinsel attempted the same technique, but found the learnt theories to be too large to manually browse. Hence, they evolved a tree-query language (TAR1) to find attribute ranges that were of very different frequencies on decision tree branches that lead to different classifications. TAR2 grew out of the realization that all the TAR1 search operations could be applied to the example set directly, without needing a decision tree learner as an intermediary. TAR2 is hence much faster than TAR1 (seconds, not hours).

A core premise of this work is that, within some system, there exists a *small* number of master attribute ranges that can have a *large* impact on the overall behavior of a system. In the literature, we have much anecdotal evidence that many domains contain a small number of variables that are crucial in determining the behavior of the whole system. Concepts analogous to master attribute ranges have been called a variety of names including *master-variables* in scheduling [5]; *prime-implicants* in model-based diagnosis [25]; *backbones* in satisfiability [27]; *dominance filter-*

ing in design [12]; *minimal environments* in the ATMS [6]; and *funnels* in HT0 [17]. We predict that TAR2 would be a useful way to summarize logs of behavior seen in all these domains.

6. CONCLUSION

The specific goal of our case study is to give a software project manager a way of addressing the *minimal management question*. We have described methods to guide managers through the dense forest of possible treatments to isolate those which are likely to have minimal effect of high-level project goals, and those which are leverage points.

The more general goal of this paper was to extend data mining into early lifecycle software engineering where both data and objective knowledge is scarce. Data miners need data and when data is absent, we can generate it using stochastic search. Generating logs using stochastic search from subjective knowledge (such as our CMM2 knowledge) is particularly fruitful since each possibility generates another entry in the log. However, large logs have to be summarized using techniques such as the TAR2 treatment learner. Our experience with TAR2 has been that this process produces a very small number of sets of treatments that are likely to be most effective in meeting overall project goals.

References

- [1] I. Bratko, I. Mozetic, and N. Lavrac. *KARDIO: a Study in Deep and Qualitative Knowledge for Expert Systems*. MIT Press, 1989.
- [2] A. Cass, B. S. Lerner, E. McCall, L. Osterweil, S. M. S. Jr., and A. Wise. Little-jil/juliette: A process definition language and interpreter. In *Proceedings of the 22nd International Conference on Software Engineering (ICSE 2000)*, pages 754–757, June 2000.
- [3] S. Chulani, B. Boehm, and B. Steece. Bayesian analysis of empirical software engineering cost models. *IEEE Transaction on Software Engineering*, 25(4), July/August 1999.
- [4] L. Chung, B. Nixon, E. Yu, and J. Mylopoulos. *Non-Functional Requirements in Software Engineering*. Kluwer Academic Publishers, 2000.
- [5] J. Crawford and A. Baker. Experimental results on the application of satisfiability algorithms to scheduling problems. In *AAAI '94*, 1994.
- [6] J. DeKleer. An Assumption-Based TMS. *Artificial Intelligence*, 28:163–196, 1986.
- [7] M. Feather, H. In, J. Kiper, J. Kurtz, and T. Menzies. First contract: Better, earlier decisions for software projects. In *Submitted to the ACM CIKM 2001: the Tenth International Conference on Information and Knowledge Management*, 2001. Available from <http://tim.menzies.com/pdf/01first.pdf>.
- [8] N. E. Fenton and M. Neil. A critique of software defect prediction models. *Software Engineering*, 25(5):675–689, 1999. Available from <http://citeseer.nj.nec.com/fenton99critique.html>.

- [9] E. T. G. Bricconi, E. Di Nitto. Issues in analyzing the behavior of event dispatching systems. In *Proceedings of the 10th International Workshop on Software Specification and Design (IWSSD-10)*, San Diego, USA, November 2000.
- [10] M. Heimdahl and N. Leveson. Completeness and consistency analysis of state-based requirements. *IEEE Transactions on Software Engineering*, May 1996.
- [11] G. Hinton. How neural networks learn from experience. *Scientific American*, pages 144–151, September 1992.
- [12] J. Josephson, B. Chandrasekaran, M. Carroll, N. Iyer, B. Wasacz, and G. Rizzoni. Exploration of large design spaces: an architecture and preliminary results. In *AAAI '98*, 1998. Available from <http://www.cis.ohio-state.edu/~jj/Explore.ps>.
- [13] N. Leveson. *Safeware System Safety And Computers*. Addison-Wesley, 1995.
- [14] N. Leveson, S. Cha, and T. Shimall. Safety verification of ada programs using software fault trees. *IEEE Software*, 8(7):48–59, July 1991.
- [15] M. Mendonca and N. Sunderhaft. Mining software engineering data: A survey, September 1999. A DACS State-of-the-Art Report. Available from <http://www.dacs.dtic.mil/techs/datamining/>.
- [16] T. Menzies. Practical machine learning for software engineering and knowledge engineering. In *Handbook of Software Engineering and Knowledge Engineering*, 2001. Available from <http://tim.menzies.com/pdf/00ml.pdf>.
- [17] T. Menzies and J. Kiper. How to argue less. In *Submitted to the ACM CIKM 2001: the Tenth International Conference on Information and Knowledge Management*, 2001. Available from <http://tim.menzies.com/pdf/01jane.pdf>.
- [18] T. Menzies, J. Kiper, and Y. Hu. Machine learning for requirements engineering. In *Submitted to ASE-2001*, 2001. Available from <http://tim.menzies.com/pdf/01ml4re.pdf>.
- [19] T. Menzies and E. Sinsel. Practical large scale what-if queries: Case studies with software risk assessment. In *Proceedings ASE 2000*, 2000. Available from <http://tim.menzies.com/pdf/00ase.pdf>.
- [20] M. Paulk, B. Curtis, M. Chrissis, and C. Weber. Capability maturity model, version 1.1. *IEEE Software*, 10(4):18–27, July 1993.
- [21] M. Paulk, C. Weber, B. Curtis, and M. Chriss. *The Capability Maturity Model: Guidelines for Improving the Software Process*. Addison-Wesley, 1995.
- [22] A. Porter and R. Selby. Empirically guided software development using metric-based classification trees. *IEEE Software*, pages 46–54, March 1990.
- [23] R. Quinlan. Induction of decision trees. *Machine Learning*, 1:81–106, 1986.
- [24] R. Quinlan. *C4.5: Programs for Machine Learning*. Morgan Kaufman, 1992. ISBN: 1558602380.
- [25] R. Rymon. An se-tree-based prime implicant generation algorithm. In *Annals of Math. and A.I., special issue on Model-Based Diagnosis*, volume 11, 1994. Available from <http://citeseer.nj.nec.com/193704.html>.
- [26] C. Seidman. *Data Mining with Microsoft SQL Server*. Microsoft Press, 2000.
- [27] J. Singer, I. P. Gent, and A. Smaill. Backbone fragility and the local search cost peak. *Journal of Artificial Intelligence Research*, 12:235–270, 2000.
- [28] J. Tian and M. Zelkowitz. Complexity measure evaluation and selection. *IEEE Transaction on Software Engineering*, 21(8):641–649, Aug. 1995.
- [29] A. van Lamsweerde. Requirements engineering in the year 00: A research perspective. In *Proceedings ICSE2000, Limerick, Ireland*, pages 5–19, 2000.
- [30] A. Wise, A. Cass, B. S. Lerner, E. McCall, L. Osterweil, and J. S.M. Sutton. Using little-jil to coordinate agents in software engineering. In *Proceedings of the Automated Software Engineering Conference (ASE 2000) Grenoble, France.*, September 2000. Available from <ftp://ftp.cs.umass.edu/pub/techrept/techreport/2000/UM-CS-2000-045.ps>.